Recommender Systems - Course Project Report

Part 2 version: data, protocol and baselines; models and honest comparison

Team	Group 2
Members	Anas Uddin (153215593), Melek Chelik (153663543), Menaka Wellane Hewage (154198965)
For Part 3 track: select one	A (diversity and popularity bias)
GitHub repository link	https://github.com/theanasuddin/movielens-recsys

1. Part 1 - Data, Protocol and Baselines

1.1 Data characterization

MovieLens 1M has 1,000,209 ratings by 6,040 users of 3,706 movies (177 of the 3,883 titles in movies.dat were never rated), so 4.47% of the user-movie matrix is filled. Every user has at least 20 ratings (median 96, maximum 2,314); movies have between 1 and 3,428 (median 123.5; 114 movies have one rating). Ratings lean positive: 1 to 5 stars take 5.6%, 10.8%, 26.1%, 34.9% and 22.6% of the ratings.

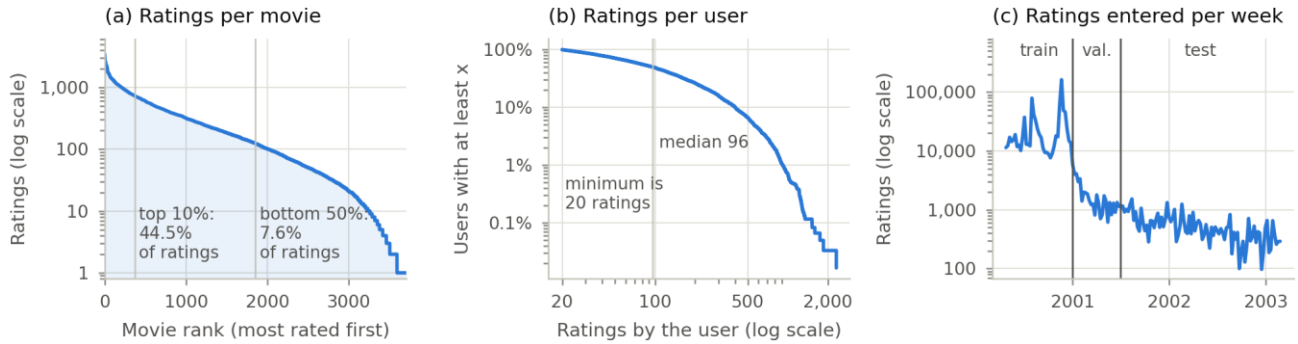


Figure 1. (a) Ratings per movie, most rated first. (b) Share of users with at least x ratings. (c) Ratings entered per week; the vertical lines are our validation and test cuts.

The long tail (Lecture 5.1, slide 5) is milder than the 80/20 rule: the 10% most-rated movies get 44.5% of the ratings, the top 20% get 65.2% and the less-rated half of the catalogue gets 7.6% (Figure 1a; Gini 0.63 over movies and 0.53 over users). Popularity also drifts. None of the ten most-rated movies of our test period is in the training top ten, eight of them are 2000 releases, and the two top-100 lists share only 54 movies.

The timestamps are the least obvious part of the data. Ratings span 25 April 2000 to 28 February 2003, but 90% were entered by 29 December 2000 (Figure 1c). Users rate in bursts when they join: 62.9% entered all their ratings within one day, and 77.4% of ratings share their exact second with another rating by the same user. A timestamp marks when a page of ratings was submitted, not when a film was watched, so the order within a session, and with it any leave-last-one-out split, is undefined. Only 799 users (13%) rate anything in our test period, and the 777 we evaluate have a median of 212 earlier ratings: the test measures heavy, returning users.

1.2 Preprocessing and feedback treatment

We parse the .dat files ourselves and check the rating range, duplicate pairs and ids; no rows are removed. The data already keeps only users with at least 20 ratings, and removing rare movies would only shrink the candidate set, so the main protocol applies no threshold (Section 1.6 shows what thresholds would have changed). Since the task is top-10 ranking (Lecture 3.1, slide 6), a held-out rating is relevant only if it is a 4 or a

5 (57.5% of all ratings); a 1 to 3 means the user saw the film and did not think much of it. Whether the models learn from all ratings as implicit interactions or only from the 4s and 5s is tuned on validation (Lecture 1.1, slide 7; Lecture 1.2, slide 11).

1.3 Train/validation/test split

We use one global temporal split (Lecture 3.1, slide 4): ratings before 1 January 2001 (UTC) train, ratings from 1 January to 30 June 2001 validate, and ratings from 1 July 2001 onwards test: 904,757 (90.5%), 47,314 (4.7%) and 48,138 (4.8%) ratings. The cuts are strict, so ratings with the same timestamp never fall on both sides. The common 80/10/10 split by count would cut on 2 December 2000, in the middle of the sign-up wave: 54% of its validation users have no earlier rating, and only 17% of its validation ratings come more than 30 days after the user's previous rating, against 92% in its test period. Validation would then measure new users and test returning ones. Our cuts put both periods in the returning-user regime: fewer than 1% of users are new, and 84% (validation) and 97% (test) of held-out ratings come more than 30 days after the user's previous one. The cost is small held-out sets: we evaluate the 892 validation and 777 test users with at least one relevant rating and some history (the test drops 19 users without a 4 or 5 and 3 without history).

1.4 Evaluation protocol

Hyperparameters are chosen by NDCG@10 on validation only; the chosen settings are refitted on training plus validation data and scored once on test (Lecture 3.1, slide 10). Each model ranks all 3,688 movies rated before 1 July 2001, minus those the user has rated, with no sampled negatives, since sampling can reorder models (slide 5; Krichene and Rendle, 2020). The 25 test ratings (0.05%) on movies nobody rated before the cut stay in the ground truth as misses, and ties are broken by a fixed random order of movies. We report Precision@10, Recall@10 (hits over the user's relevant test movies) and NDCG@10 with binary gains (slides 7-8), averaged over users, and catalogue coverage, the share of candidates in at least one top-10 list (slide 9). Our metric code reproduces the example on slide 8 and matches scikit-learn's ndcg_score on 500 random rankings to within 1e-12. Intervals are 95% bootstrap intervals over users (1,000 resamples; Efron and Tibshirani, 1993), paired comparisons add a Wilcoxon signed-rank test, and all randomness uses seed 42.

1.5 Baseline methods and settings

Most popular gives every user the same list, ranked by the number of training ratings, minus what they have seen (Lecture 1.1, slide 9). We try eight variants: all training ratings or only the last 30, 90 or 180 days, counting every rating or only the 4s and 5s. Item-based kNN (Lecture 1.2, slides 5-11) keeps for each movie i its k most similar movies with a positive similarity and scores i for a user by summing the similarities of those neighbours the user has rated (Deshpande and Karypis, 2004). We tune the similarity (cosine or Jaccard on the binary matrix, the lecture's defaults for implicit data, or adjusted cosine on the stars; Sarwar et al., 2001), the training signal for cosine and Jaccard, and k from 10 to 1,000: 35 kNN and 8 popularity settings in all.

Validation chose cosine on ratings of 4 and 5 with k = 500 (Figure 2). The training signal mattered most: using only positive ratings lifts the best cosine setting from 0.140 to 0.174 and Jaccard from 0.129 to 0.166, because otherwise a 1-star rating counts as evidence that two films belong together. NDCG@10 rises with k up to 200-500, well above the 20-50 the lecture calls typical (slide 10), but coverage falls from 18.7% at k = 10 to 10.7% at k = 500: with summed similarities, a larger neighbourhood pulls the ranking toward popular movies. Adjusted cosine peaks at k = 50. The best settings are close (Jaccard with k = 200 is 0.008 behind), so the similarity measure matters less than the signal and k. For most popular, validation chose the last 30 days of positive ratings.

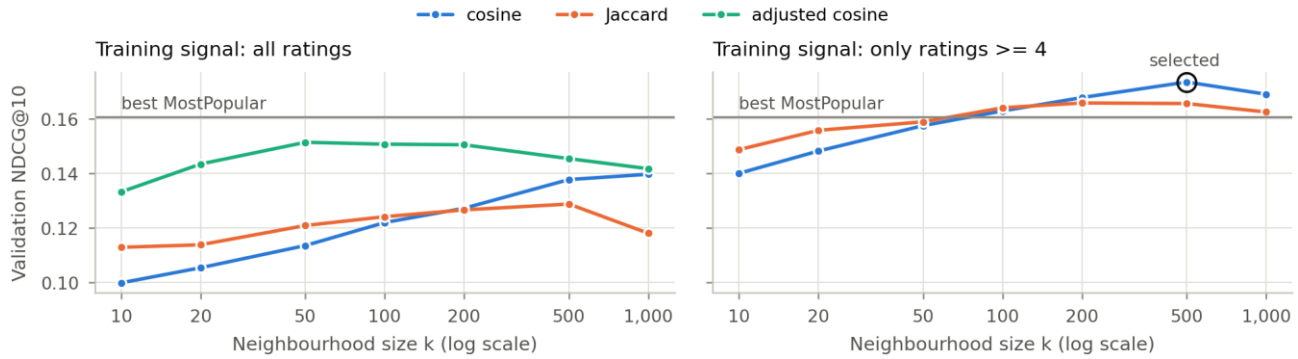


Figure 2. Validation NDCG@10 of item-based kNN against k, per similarity measure and training signal. The grey line is the best most-popular variant on validation.

1.6 Results and analysis

Table 1. Test results for 777 users and 3,688 candidate movies, with validation NDCG@10 for comparison. Bold rows are the settings chosen on validation. All 14 variants are in the results workbook.

Model and settings	Val. NDCG@10	P@10	R@10	NDCG@10 [95% CI]	Coverage
Random (seed 42)	-	0.012	0.002	0.012 [0.009, 0.015]	86.8%
Most popular: all-time, all ratings	0.135	0.153	0.057	0.169 [0.154, 0.185]	4.1%
Most popular: all-time, ratings ≥ 4	0.144	0.159	0.062	0.177 [0.162, 0.193]	4.0%
Most popular: last 30 days, ratings ≥ 4	**0.161**	**0.122**	**0.070**	**0.159 [0.145, 0.172]**	**2.0%**
Most popular: last 180 days, ratings ≥ 4	0.143	0.148	0.089	0.176 [0.161, 0.189]	2.7%
Item kNN: cosine, ratings ≥ 4, k = 500	**0.174**	**0.183**	**0.072**	**0.207 [0.192, 0.225]**	**9.8%**
Item kNN: Jaccard, ratings ≥ 4, k = 200	0.166	0.179	0.075	0.202 [0.186, 0.219]	9.8%
Item kNN: adjusted cosine, all ratings, k = 50	0.151	0.170	0.071	0.189 [0.174, 0.205]	11.8%
Item kNN: cosine, all ratings, k = 1000	0.140	0.149	0.057	0.167 [0.152, 0.181]	10.6%

Item-based kNN is the stronger baseline (Table 1). Its test NDCG@10 of 0.207 beats the selected popularity variant by 0.049 (95% CI [0.031, 0.068], Wilcoxon $p < 0.001$; better for 374 users, worse for 219) and the all-time list by 0.039 [0.028, 0.050], with 9.8% coverage against 1.6-4.1% for popularity. The gain comes from precision and ranking; against the selected popularity variant the Recall@10 difference is only 0.002 [-0.012, 0.015].

The popularity results expose a flaw we did not expect. The 30-day window that won on validation is worse on test than the all-time count of positive ratings (-0.018 [-0.036, -0.002]). A window in days is a different object at each cut: the 30 days before 1 January 2001 hold 107,641 ratings from 1,210 users, mostly new members, against 4,981 ratings from 271 users before 1 July 2001. Recent windows still have the best recall (0.064-0.089 against 0.057-0.062 for all-time counts), since they catch the 2000 releases that returning users go on to rate. A window defined by a number of ratings would mean the same at both cuts; we will test this in Part 2.

Filtering rare movies changes almost nothing. Removing movies with fewer than 5, 10, 20 or 50 training ratings deletes 0.06% to 2.4% of training rows and leaves as few as 2,465 candidates, yet moves NDCG@10 by less than 0.002, even when their test ratings are dropped too: neither baseline recommends these movies. Evaluating only users with 50 or more training ratings matters more: kNN loses 0.002, popularity gains 0.008 and the gap falls from 0.049 to 0.039. No threshold moved the results by more than the gap, but the user threshold moves the gap itself, a hint that the baselines serve light and heavy users differently (to be broken down in Part 2).

The split matters most. With the 80/10/10 split, validation chose nearly the same settings (k = 1000; a 90-day window), and they score within 0.001 of ours on our test set. On that split's own test set, however, kNN reaches 0.246 and popularity 0.211. Moving the cut shifted the numbers by 0.04-0.05, as much as the gap

between the baselines, while their order held. Results from different splits, including other teams', are not directly comparable.

2. Part 2 - Models and Honest Comparison

2.1 Models

We add five models to the two Part 1 baselines (Table 2). They cover five paradigms of the brief: user-based kNN (neighbourhood CF), biased matrix factorisation (latent-factor CF), BPR-MF (ranking-loss CF), EASE, a linear item-item autoencoder (Lecture 3.2, slides 9 and 11; Steck, 2019), and content-based profile matching over genres and release decade (Lecture 2.2, slides 4-7). All models are our own numpy code and run through the Part 1 evaluator, so they share the candidate set, the seen-movie rule and the metrics.

Biased MF and BPR-MF are the two models we designate as self-implemented. Biased MF predicts $r(u, i) = \mu + b(u) + b(i) + p(u)\cdot q(i)$ and takes mini-batch SGD steps on the squared error of the observed ratings, with separate L2 penalties for factors and biases (Lecture 2.1, slides 4 and 7-9; Koren et al., 2009); inside a batch, a repeated user or movie averages its gradients. BPR-MF scores $s(u, i) = p(u)\cdot q(i) + b(i)$ and ascends $\ln \sigma(s(u, i) - s(u, j))$ over sampled triples in which u rated i but not j (Lecture 2.1, slide 12; Rendle et al., 2009); j is drawn uniformly from the candidates and drawn again if it is in u's training signal. Our tests compare both models' gradient code with finite differences and check that each recovers planted structure in synthetic data; EASE is checked against per-column ridge regression and user kNN against brute-force voting.

Table 2. The seven models: validation trials, the setting chosen on validation, validation and test NDCG@10, and test catalogue coverage. * marks the two self-implemented models; bold is the best model on validation. Test values of the two stochastic models are means over five seeds.

Model	Paradigm	Trials	Chosen setting	Val.	Test	Cov.
Most popular	Non-personalised, baseline	16	last 30,000 ratings, ratings ≥ 4	0.169	0.167	2.2%
Item kNN	Neighbourhood CF, baseline	35	cosine, ratings ≥ 4, k = 500	0.174	0.207	9.8%
User kNN	Neighbourhood CF	24	cosine, ratings ≥ 4, k = 500	0.170	0.209	6.5%
Biased MF*	Latent-factor CF	24	64 factors, lr 0.012, reg 0.019, bias reg 0.56, 25 epochs	0.076	0.101	25.2%
BPR-MF*	Ranking-loss CF	24	32 factors, lr 0.012, reg 8.5e-03, ratings ≥ 4, 40 epochs	0.178	0.206	17.9%
EASE	**Linear autoencoder**	**18**	**λ = 5000, ratings ≥ 4**	**0.178**	**0.218**	**10.0%**
Content profile	Content-based	12	mean-centred profile, decade weight 1, dot match	0.023	0.032	30.3%

2.2 Hyperparameter tuning and variability

Every model, the baselines included, is tuned the same way: each setting is fitted on the training period and scored by NDCG@10 on validation, and the best setting is refitted on training plus validation for one test run (Lecture 3.1, slide 10). Models with a few discrete settings get their whole grid (16 settings for most popular, the 35 of Part 1 for item kNN, 24 for user kNN, 18 for EASE and 12 for the content profile). Biased MF and BPR-MF get 24 seeded random draws over factors, learning rate and penalties, and inside each draw choose their number of epochs: validation is checked every 5 epochs, and training stops after two checks without improvement or at 40. In total 153 validation trials ran; the search spaces are in configs/default.yaml and every trial is in results/tables/part2_validation_trials.csv. The chosen BPR-MF draw was still improving at 40 epochs, so it may be slightly under-trained.

Each model gets a 95% bootstrap interval over the 777 test users. Biased MF and BPR-MF are refitted with five seeds; their per-user metrics are averaged over the seeds, and the spread between seeds is reported next to

the user interval. The best model on validation, EASE, is compared with every other model on paired per-user NDCG@10, with a bootstrap interval for the difference and a Wilcoxon signed-rank test, Holm-corrected over the six comparisons (Holm, 1979).

2.3 Overall results

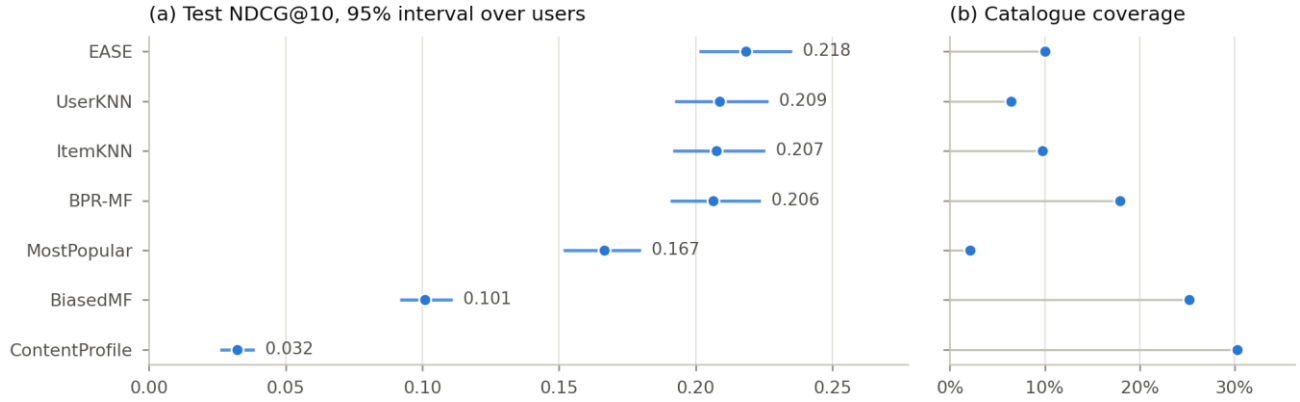


Figure 3. (a) Test NDCG@10 of the seven models with 95% bootstrap intervals over the 777 test users; biased MF and BPR-MF are averaged over five seeds. (b) Share of the candidate movies that appear in at least one test list.

EASE is the best model on validation and on test, with a test NDCG@10 of 0.218 (Table 2, Figure 3). Its lead over item kNN, user kNN and BPR-MF is small, 0.010 to 0.012, but every paired comparison finds it (Holm-corrected $p \leq 0.007$); against item kNN, EASE does better for 286 users and worse for 216. The three runners-up lie within 0.002 of each other, less than BPR-MF moves between its five seeds (0.203 to 0.211), so we do not rank them. Most popular trails EASE by 0.052 [0.034, 0.070] in NDCG@10, but its Recall@10 is no lower: EASE minus most popular is -0.006 [-0.022, 0.009]. As with item kNN in Part 1, the gain is in precision and in the order of the list.

Biased MF is the instructive failure. As a rating predictor it works: its test RMSE of 0.928 to 0.930 beats the item mean (0.968) and the global mean (1.103). As a ranker it reaches 0.101, below most popular. Its loss only measures error on ratings that exist, and sorting by predicted rating favours well-rated movies that few people have seen: 63% of its slots go to tail movies and 6.3% to movies with fewer than 20 training ratings, against 2.2% and 0.1% for EASE. This is the gap between rating error and top-N accuracy that Lecture 2.1 (slide 11) warns about and Cremonesi et al. (2010) measured. BPR-MF fits the same kind of factor model with a loss that compares a rated movie with an unrated one, which is the ranking task itself, and scores 0.206. The content profile comes last (0.032): two movies with the same genres and decade are the same movie to it (Lecture 2.2, slide 9), and a mature film catalogue with dense logs is where the lecture expects CF to lead (slide 12).

Part 1 left a question open: would a popularity window of a fixed number of ratings carry over from the validation cut to the test cut better than a window of days? Validation, now also offered the 10,000 to 300,000 most recent ratings, chose the last 30,000 ratings of 4 and 5 (0.169). On test this scores 0.167, above the 30-day window (0.159) but below the count of all positive ratings (0.177); these test scores are a check and chose nothing. A fixed count does not make the two cuts alike either: the last 30,000 ratings span 14 days before the validation cut and 152 days before the test cut, because far fewer ratings were entered in 2001 (results/tables/part2_popularity_windows.csv).

2.4 Disaggregated analysis

Table 3. Test NDCG@10 overall and per group; bold is the best model in each column. Light, medium and heavy: thirds of the test users by training ratings (≤ 129, 129-319 and > 319; 261, 259 and 257 users). Head and tail: only the 369 most-rated movies (at least 700 ratings) or only the other 3,319 candidates count as relevant; the lists stay the same. Early and late: test ratings before or from 3 February 2002, the median test date.

Model	All	Light	Medium	Heavy	Head	Tail	Early	Late
Most popular	0.167	0.133	0.155	0.213	0.142	**0.068**	0.142	0.105
Item kNN	0.207	0.211	0.205	0.207	0.242	0.005	0.148	0.152
User kNN	0.209	0.217	0.206	0.202	0.249	0.001	0.149	0.153
Biased MF	0.101	0.083	0.104	0.116	0.087	0.031	0.073	0.074
BPR-MF	0.206	0.216	0.193	0.210	0.227	0.020	0.143	0.155
EASE	**0.218**	**0.218**	**0.211**	**0.227**	**0.259**	0.004	**0.154**	**0.161**
Content profile	0.032	0.036	0.026	0.035	0.018	0.019	0.024	0.021

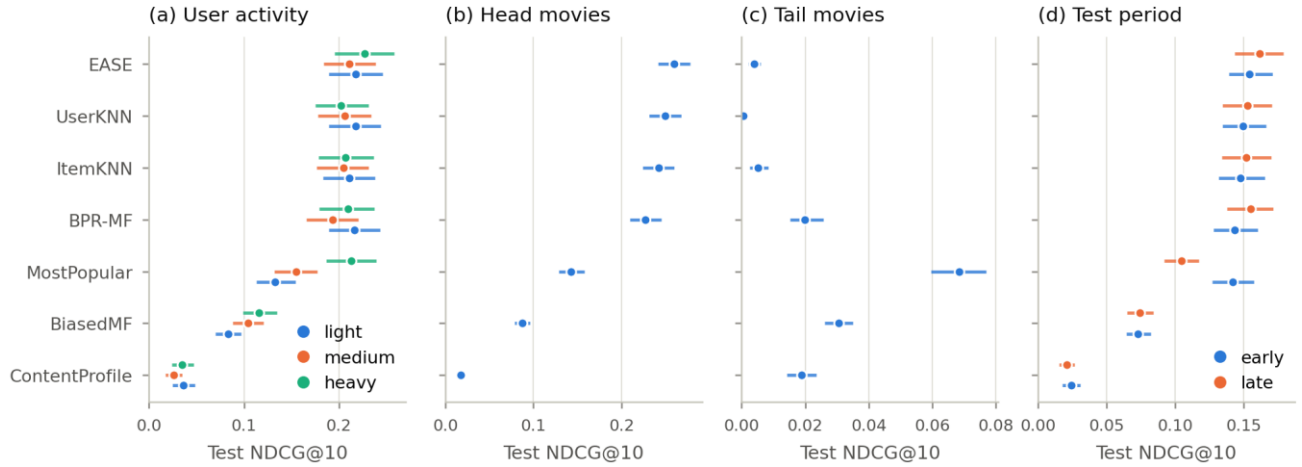


Figure 4. The values of Table 3 with 95% bootstrap intervals over the users in each group. Panel (c) has its own scale.

Table 3 and Figure 4 break the test results down by user activity, item popularity and test period (Lecture 3.1, slide 9; Lecture 5.2, slide 6). Light users are not worse off under the four strongest models: EASE scores 0.218, 0.211 and 0.227 for light, medium and heavy users. Activity matters for most popular, which rises from 0.133 to 0.213 (biased MF from 0.083 to 0.116), so EASE's lead over most popular shrinks from 0.085 [0.049, 0.120] for light users to 0.014 [-0.015, 0.048] for heavy users, where the interval includes zero. This is the difference between light and heavy users that Part 1 pointed to (Section 1.6).

By item popularity the ranking turns over. On the 369 head movies EASE is the best model (0.259). On the 3,319 tail movies it scores 0.004, about as little as item kNN (0.005) and user kNN (0.001), while most popular reaches 0.068, biased MF 0.031, BPR-MF 0.020 and even the content profile 0.019; each of the four beats EASE on the tail by a paired interval that excludes zero. The tail is not a corner of the test set: 57% of the relevant test ratings are on tail movies. A model fitted mostly on the head predicts the tail near its baseline, and accuracy over all ratings rewards that (Lecture 5.1, slide 5). Most popular reaches the tail through recency: 97% of its tail slots are 2000 releases, which hold 9% of the relevant tail ratings.

By test period, most popular falls from 0.142 before 3 February 2002 to 0.105 after it, as its counts age, while EASE holds (0.154 and 0.161); EASE's lead over most popular grows from 0.012 [-0.007, 0.032] to 0.057 [0.036, 0.078]. So EASE, our headline winner, has the highest score in every user group and in both halves of the test period, but is close to useless on tail movies, which hold most of the relevant test ratings.

2.5 Sparsity ablation

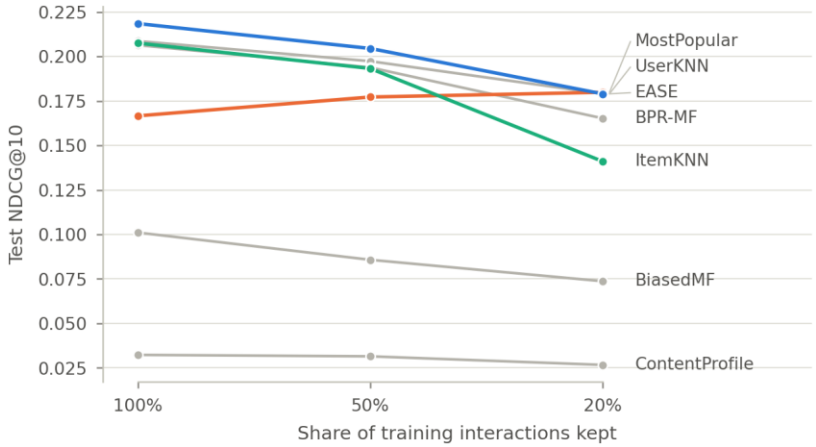


Figure 5. Test NDCG@10 with 100%, 50% and 20% of each user's ratings, means over three samples (standard deviations at most 0.005). The full history stays out of the lists at every level.

For the ablation we keep 50% or 20% of each user's ratings (at least one), re-tune every model on a sample of the training period with 8 trials (its full-data setting and seven random draws from its search space), and test on three samples of training plus validation. The task does not change: the full history still decides which movies a user has seen and which movies are candidates, so only the training signal shrinks. Our first version also removed the dropped ratings from the seen set, so the lists filled up with movies the users had in fact rated, and item kNN seemed to fall from 0.202 to 0.080 at 50% (commit ec678ea).

EASE keeps its lead at 50% (0.204 against 0.197 for user kNN; paired difference 0.007 [0.003, 0.011]) but not at 20%, where most popular, user kNN and EASE lie within 0.001 of each other (0.180, 0.179 and 0.179); the paired test of the first two finds no difference (0.001 [-0.007, 0.009], p = 0.397; Figure 5). With a fifth of the data, the best personalised models no longer beat a popularity list. From 100% to 20%, user kNN loses 14% of its NDCG@10, EASE 18% and BPR-MF 20%. Item kNN, the stronger Part 1 baseline, loses 32% and falls from third to fifth place, while its coverage jumps from 9.8% to 27.5%. Sparsity is the first problem Lecture 1.2 (slide 12) lists for neighbourhood methods; in our data it hits item kNN far harder than user kNN.

Most popular is the one model that gains when data are removed (0.167, 0.177 and 0.180), and the cause is its window. Re-tuning kept the last 30,000 positive ratings at every level. Keeping a fifth of each user's ratings thins the rating stream about fivefold, so on the 20% samples these ratings reach back over roughly the period of the last 150,000 full-data ratings, and on full data such longer windows do better on test (0.189 for the last 100,000 positive ratings, 0.182 for 300,000). The gain comes from the mismatch between the cuts seen in Section 2.3, not from having less data.

Re-tuning at each level matters. Biased MF with its full-data setting frozen falls to 0.034 at 20% on the first sample; re-tuning picks the same setting trained for 5 epochs instead of 25 and keeps 0.079. For BPR-MF the eight trials picked a setting that does worse than the frozen one (0.160 against 0.169 at 20%): a small budget is noisy too, so the 20% result may understate BPR-MF.

3. Part 3 - Beyond Accuracy

3.1 Selected track and motivation

We take Track A, diversity and popularity bias (Lecture 5.1). Part 2 shows why. The most accurate models recommend a narrow, popular slice of the catalogue: EASE, item kNN and user kNN cover 6.5% to 10.0% of

the candidates and give at most 2.8% of their slots to tail movies, although 57% of the relevant test ratings are on tail movies, while the models that reach further, biased MF and the content profile, lose most of the accuracy. In Part 3 we will re-rank the top 100 of EASE, the best Part 2 model, with maximal marginal relevance (slide 9). Items are the 18-genre vectors of MovieLens and the item distance is 1 minus their cosine similarity. We will sweep the trade-off λ and report NDCG@10 against intra-list diversity, catalogue coverage, the Gini index of recommendation counts and the calibration error of the genre mix, as a curve (slides 7 and 13).

3.2 Method

3.3 Results and trade-offs

3.4 Cost and interpretation

3.5 "One number we cannot explain"

4. Discussion and Conclusions

5. Contribution Statement

Anas Uddin: set up the repository and the evaluation code, ran the Part 1 experiments and drafted Sections 1.1-1.6. In Part 2, Anas reworked and extended the models and the experiment runner, added BPR-MF and EASE, fixed the sparsity protocol, ran the final experiments and drafted Sections 2.1-2.5 and 3.1. Anas also worked through the requirements and shaped the solution together with Melek and Menaka.

Melek Chelik: took an equal share of the implementation. With the team, Melek worked through the requirements and helped derive and steer the solution, including the Part 1 protocol and baselines. In Part 2, Melek wrote the first implementation that the final version builds on: user kNN, biased MF, the content profile, tuning with an equal budget, the breakdowns and the sparsity ablation.

Menaka Wellane Hewage: contributed equally to the implementation. Menaka took part in the team's discussions of the requirements and helped brainstorm, work out and guide the solution with us.

6. Use of AI Assistants

We used Claude (Anthropic) in the Claude Code desktop app to plan the repository, write first versions of the code and unit tests, review and rework the Part 2 code, run the experiments and draft the report text from the generated result tables. We read and tested the code, checked the reported numbers against results/results.csv and results/tables, and edited the text; the team is responsible for all of it.

References

Cremonesi, P., Koren, Y. and Turrin, R. (2010). Performance of recommender algorithms on top-N recommendation tasks. In Proceedings of RecSys 2010, 39-46.

Deshpande, M. and Karypis, G. (2004). Item-based top-N recommendation algorithms. ACM Transactions on Information Systems, 22(1), 143-177.

Efron, B. and Tibshirani, R. J. (1993). An Introduction to the Bootstrap. Chapman and Hall.

Harper, F. M. and Konstan, J. A. (2015). The MovieLens datasets: History and context. ACM Transactions on Interactive Intelligent Systems, 5(4), Article 19.

Holm, S. (1979). A simple sequentially rejective multiple test procedure. Scandinavian Journal of Statistics, 6(2), 65-70.

Koren, Y., Bell, R. and Volinsky, C. (2009). Matrix factorization techniques for recommender systems. Computer, 42(8), 30-37.

Krichene, W. and Rendle, S. (2020). On sampled metrics for item recommendation. In Proceedings of KDD 2020, 1748-1757.

Rendle, S., Freudenthaler, C., Gantner, Z. and Schmidt-Thieme, L. (2009). BPR: Bayesian personalized ranking from implicit feedback. In Proceedings of UAI 2009, 452-461.

Sarwar, B., Karypis, G., Konstan, J. and Riedl, J. (2001). Item-based collaborative filtering recommendation algorithms. In Proceedings of WWW 2001, 285-295.

Steck, H. (2019). Embarrassingly shallow autoencoders for sparse data. In Proceedings of The Web Conference 2019, 3251-3257.

Stefanidis, K. (2026). DATA.ML.360 Recommender Systems, lecture slides 1.1-6.2. Tampere University.